

workflow-guide

Superspec Workflow Guide

This guide provides detailed phase-by-phase instructions for the superspec development workflow. The SKILL.md file references this document for progressive disclosure — the agent reads relevant sections as needed during command execution.

Phase 0: Project Initialization

Command: /speckit.constitution **Gate:** None — this is the entry point. **Output:** .specify/ directory structure + memory/ constitution.md

Steps

1. **Create directory structure:**

```
.specify/  
├─ memory/  
├─ specs/  
└─ templates/
```

2. **Copy templates** from the superspec skill's templates/ directory into .specify/templates/. This gives the project its own copy of templates that can be customized.

3. **Interview the user** about:

- Project name and purpose
- Core principles (3-7 principles, each with a clear name and description)
- Technology stack (layers, technologies, purposes)
- Development workflow preferences (which quality gates to enforce)
- Governance rules

4. **Generate constitution** using constitution-template.md as skeleton. Fill in all sections based on user responses. Ensure the “Development Workflow” and “Quality Gates” sections are populated.
5. **Write** to .specify/memory/constitution.md

Verification

- File exists at .specify/memory/constitution.md
 - All template placeholders replaced with actual content
 - At least 3 core principles defined
 - Quality gates section has at least one requirement marked
-

Phase 1: Specification

Command: /speckit.specify **Gate:** Constitution must exist at .specify/memory/constitution.md **Output:** specs/NNN-feature-name/spec.md

Steps

1. **Verify constitution** exists. If not, guide user to run /speckit.constitution first.
2. **Determine spec number:** Scan specs/ for existing directories. Next number = highest existing + 1, zero-padded to 3 digits (001, 002, ...).
3. **Read constitution** to understand project constraints and principles.
4. **Interview the user** about:
 - Feature name and high-level description
 - User scenarios (who does what, why, expected outcomes)
 - Priority ranking of scenarios (P1, P2, P3)
 - Known requirements and constraints
 - Success criteria (measurable outcomes)
 - Assumptions
5. **Generate spec** using spec-template.md as skeleton.
 - Fill in user stories with Given/When/Then acceptance scenarios
 - Mark functional requirements with MUST/SHOULD/MAY
 - Flag unclear items as [NEEDS CLARIFICATION]

- Leave “Open Questions”, “Brainstorm Log”, and “Brainstorm Prompts” sections with initial prompts but no resolved content

6. **Write** to specs/NNN-feature-name/spec.md

7. **Suggest next step:** “Run /speckit.superspec.brainstorm specs/NNN-feature-name/spec.md to discover edge cases before planning.”

Verification

- File exists at expected path
 - At least one user story with acceptance scenarios
 - Priority assigned to each user story
 - Functional requirements section populated
 - Success criteria section populated
-

Phase 2: Brainstorming

Command: /speckit.superspec.brainstorm **Gate:** Target spec file must exist. **Output:** Updated spec file (edge cases, open questions, brainstorm log).

Superpowers Integration

If the brainstorming skill is detected (see [superpowers-bridge.md](#)):
- Read its SKILL.md and follow its questioning protocol - Adapt outputs to update the spec file instead of creating separate documents - Follow its session structure (one question at a time, multiple choice when possible)

Built-in Fallback Protocol

When superpowers brainstorming is not available, use this 5-category questioning protocol:

Category 1: Boundary Conditions

Ask about minimum/maximum values, empty states, and edge-of-range inputs. - “What happens when [input] is empty?” - “What’s the maximum number of [items] the system should handle?” - “What happens at exactly the boundary of [limit]?”

Category 2: Error Scenarios

Ask about failure modes, recovery, and degradation. - "What happens when [external service] is unavailable?" - "How should the system recover from [failure type]?" - "What error message should the user see when [scenario]?"

Category 3: Scale & Performance

Ask about load, concurrency, and resource constraints. - "What happens with [N]x expected traffic?" - "Are there rate limits needed for [operation]?" - "What's the acceptable response time for [action]?"

Category 4: Security & Privacy

Ask about attack vectors, data protection, and authorization. - "Can [feature] be abused by [actor type]?" - "What data needs to be encrypted or redacted?" - "Who should NOT have access to [resource]?"

Category 5: User Experience

Ask about confusion points, accessibility, and unintended usage. - "What if the user tries to [unintended action]?" - "How does this work for users with [accessibility need]?" - "What happens if the user navigates away mid-process?"

Process

1. **Read the spec** and identify areas with thin coverage or placeholder edge cases.
2. **Ask ONE question at a time.** Wait for the user's answer before proceeding. Prefer multiple-choice format when possible for efficient exploration.
3. **After each answer:**
 - If it reveals a new requirement: add to spec's Functional Requirements
 - If it resolves an open question: update the Open Questions table
 - If it reveals an edge case: add to the Edge Cases section
 - If it changes an acceptance scenario: update the relevant user story
4. **Continue** through all 5 categories, skipping questions already covered by the spec.

5. **When the user says the spec is ready** (or all categories explored):

- Add a dated entry to the “Brainstorm Log” section
- Summarize: number of questions explored, insights discovered, spec updates made
- Suggest: “Run /speckit.plan to create the implementation plan.”

Iteration

This phase can be run multiple times. Each session: - Starts by reading previous brainstorm log entries to avoid repeating questions - Focuses on the user-specified topic (if provided) or unexplored categories - Appends a new entry to the brainstorm log

Phase 3: Planning

Command: /speckit.plan **Gate:** Spec file must exist. Brainstorm is recommended but not required. **Output:** specs/NNN-feature-name/plan.md

Steps

1. **Read inputs:** spec file, constitution, any previous plan for this feature.
2. **Constitution compliance check:** Verify the planned approach respects every principle. Document results in the “Constitution Check” table.
3. **Research the codebase:** Use Glob, Grep, and Read tools to determine:
 - Language/framework already in use
 - Existing patterns and conventions
 - Files that will need modification
 - Testing framework in use
4. **Design project structure:** Determine which files to create or modify. Document in the “Source Code” section of the plan.
5. **Determine execution strategy:**
 - Which components need TDD? (complex logic, critical paths)
 - Which work streams can run in parallel? (independent files)

- Where are human checkpoints needed? (before integration, before merge)
 - What needs code review? (APIs, security, data models)
6. **Superpowers integration:** If writing-plans skill is detected, read it and use its blueprint process to enhance the execution strategy section.
 7. **Generate plan** using plan-template.md as skeleton. Fill all sections.
 8. **Write** to specs/NNN-feature-name/plan.md

Verification

- Constitution check table completed — no unresolved violations
 - Technical context filled (no remaining NEEDS CLARIFICATION without good reason)
 - Project structure documented with real file paths
 - Execution strategy populated with at least TDD and checkpoint decisions
-

Phase 4: Task Decomposition

Command: /speckit.superspec.tasks **Gate:** Plan must exist for the target feature. **Output:** specs/NNN-feature-name/tasks.md

Steps

1. **Read inputs:** plan, spec, constitution.
2. **Superpowers integration:** If writing-plans skill is detected, read it and follow its task decomposition process, but structure the output using tasks-template.md format.
3. **Decompose** the plan into phases:
 - Phase 1: Setup (project structure, dependencies)
 - Phase 2: Foundational (blocking prerequisites)
 - Phase 3+: One phase per user story, ordered by priority
 - Final phase: Polish and cross-cutting concerns
4. **Apply execution markers** from the plan's execution strategy:
 - [TDD] for components identified as needing test-first approach
 - [REVIEW] for components identified as needing review gates
 - [SUBAGENT] for independent work streams
 - [P] for tasks within the same phase that can run in parallel

5. **Define checkpoints** at each phase boundary.
6. **Document dependencies** and execution order.
7. **Write** to specs/NNN-feature-name/tasks.md

Verification

- Every user story from the spec has corresponding tasks
 - Tasks are traceable to user stories via [US#] labels
 - Execution markers match the plan's execution strategy
 - Phase dependencies are documented
 - At least one checkpoint per phase transition
-

Phase 5: Execution

Command: `/speckit.superspec.execute` **Gate:** Tasks file must exist for the target feature. **Output:** Code changes, updated task checkboxes.

Steps

1. **Read inputs:** tasks, plan, spec, constitution.
2. **Superpowers detection:** Check for executing-plans, subagent-driven-development, and test-driven-development skills.
3. **Walk through tasks** phase by phase, respecting execution markers:

For [TDD] tasks:

- Write the test first
- Run it — verify it FAILS
- Implement the minimum code to pass
- Run it — verify it PASSES
- Refactor if needed
- If TDD skill available: follow its full process

For [SUBAGENT] tasks:

- If subagent skill available: follow its dispatch protocol
- Otherwise: implement sequentially
- If marked [P] as well: use the Task tool for parallel execution

For [REVIEW] tasks:

- Complete the implementation

- Present the changes to the user
- Wait for explicit approval before continuing

For [P] tasks:

- Launch parallel tasks using the Task tool where possible
- Ensure no dependency conflicts between parallel tasks

4. At each checkpoint:

- Summarize completed work
- Run applicable tests
- Report results
- Ask: “Phase [N] complete. Proceed to Phase [N+1]?”
- Wait for explicit user approval

5. Update task checkboxes in `tasks.md` as each task completes.

Human Checkpoint Protocol

The agent MUST: - Never skip a phase checkpoint - Present a clear summary of what was done - Report test results if tests were run - Wait for explicit “proceed” or “continue” from the user - If the user requests changes, address them before proceeding

Phase 6: Review

Command: `/speckit.superspec.review` **Gate:** Implementation must exist (at least some tasks completed). **Output:** Review findings reported to user.

Steps

1. **Read inputs:** spec (acceptance scenarios), plan (constitution check), constitution (principles).
2. **Superpowers detection:** If requesting-code-review skill is available, follow its review protocol.
3. **Review dimensions** (built-in protocol):
 1. **Spec compliance:** For each acceptance scenario in the spec, verify it is implemented and can be demonstrated.
 2. **Edge case coverage:** For each edge case in the spec (including those from brainstorming), verify handling exists.

3. **Constitution compliance:** For each principle, verify the implementation respects it.

4. **Code quality:** Check for correctness, security, error handling, performance.

5. **Test coverage:** Verify tests exist for critical paths.

4. **Report findings** with:

- Confidence score (0-100, only report issues ≥ 80)
- Severity (Critical / Important / Suggestion)
- File path and line reference
- Specific recommendation

5. **Group** by severity, highest first.

Quick Reference

Phase	Command	Gate	Output
0	/speckit.constitution	None	.specify/ memory/ constitution.md
1	/speckit.specify	Constitution exists	specs/NNN/ spec.md
2	/speckit.superspec.brainstorm	Spec exists	Updated spec.md
3	/speckit.plan	Spec exists	specs/NNN/ plan.md
4	/speckit.superspec.tasks	Plan exists	specs/NNN/ tasks.md
5	/speckit.superspec.execute	Tasks exist	Code + updated tasks.md
6	/speckit.superspec.review	Implementation exists	Review report

Session Resumability

Superspec is designed to survive session interruptions. All state lives in plain-text files under .specify/memory/ (governance — constitution.md) and specs/NNN-*/ (per-feature — spec.md, plan.md, tasks.md, progress.yml). This section documents how the agent detects and resumes work.

Progress File: progress.yml

Each feature spec directory may contain a progress.yml file:

```
feature: user-authentication
created: 2026-04-22
current_phase: execute
phases:
  constitution: { status: done, updated: 2026-04-22 }
  specify:      { status: done, updated: 2026-04-22 }
  brainstorm:   { status: done, updated: 2026-04-22, sessions: 2 }
  plan:         { status: done, updated: 2026-04-22 }
  tasks:        { status: done, updated: 2026-04-22 }
  execute:      { status: in_progress, updated: 2026-04-22,
                  current_task: T012, completed_tasks: 11, total_tasks: 19 }
  review:       { status: pending }
```

Resume Check Protocol

Every superspec command begins with:

1. **Scan .specify/** — does it exist? Are there spec directories?
2. **Read superpowers.yml** — which superpowers skills are available? If the file does not exist, run detection and create it.
3. **Read progress.yml** — what phase is each feature in?
4. **If no progress.yml:** Infer progress from file existence:
 - constitution.md → constitution done
 - spec.md → specify done
 - spec.md has Brainstorm Log entries → brainstorm was run at least once
 - plan.md → plan done
 - tasks.md → tasks done
 - tasks.md has [x] checkboxes → execute in progress
5. **Report** current state to the user (including superpowers status) before proceeding
6. **Resume** from the detected point (see phase-specific rules below)

Phase-Specific Resume Rules

Constitution (in_progress): - Re-read constitution.md - Identify sections still containing template placeholders ([PRINCIPLE_NAME], etc.) - Ask user about remaining sections only

Specify (in_progress): - Re-read spec.md - Look for [NEEDS CLARIFICATION] markers and empty placeholder sections - Continue the interview for unresolved items only

Brainstorm (in_progress): - Re-read spec.md Brainstorm Log to see which sessions have been completed - Re-read Open Questions table — count Open vs Resolved - Skip categories already covered in previous sessions - Resume from the first unexplored category or open question

Plan (in_progress): - Re-read plan.md - Look for NEEDS CLARIFICATION fields - Fill in missing technical context; do not regenerate completed sections

Tasks (in_progress): - Re-read tasks.md - Verify all user stories from spec.md have corresponding tasks - Add missing tasks without disrupting existing task numbering

Execute (in_progress): - Re-read tasks.md — parse checkboxes - Count [x] (completed) vs [] (remaining) - Identify the **current phase** (first phase with unchecked tasks) - Skip all completed tasks; resume from the first [] task in that phase - If a phase checkpoint was not yet confirmed, re-present the checkpoint summary

Review (in_progress): - Re-read any existing checklist files - Continue from unchecked review items

Writing progress.yml

The agent updates progress.yml at these moments:

Event	Update
Command starts	Set phase to in_progress, update timestamp
Command completes successfully	Set phase to done, update timestamp
Brainstorm session ends	Increment sessions counter
Task checkbox toggled during execute	Update completed_tasks and current_task
User explicitly skips a phase	Set phase to skipped

If progress.yml does not exist when a command runs, create it with all prior phases inferred as done based on existing files.

Superpowers Status File: superpowers.yml

A project-level file .specify/superpowers.yml persists the superpowers detection results so they are **visible in the project docs** and **don't require re-detection on every command**.

```
# .specify/superpowers.yml
last_checked: 2026-04-22T14:30:00
```

```
skills:
  brainstorming:
    detected: true
    path: ~/.agents/skills/brainstorming/SKILL.md
  writing-plans:
    detected: true
    path: ~/.agents/skills/writing-plans/SKILL.md
  executing-plans:
    detected: false
  subagent-driven-development:
    detected: false
  test-driven-development:
    detected: true
    path: .agents/skills/test-driven-development/SKILL.md
  requesting-code-review:
    detected: false
```

When to update `superpowers.yml`:

Event	Action
<code>/speckit.constitution</code> (first run)	Create the file with full detection results
<code>/speckit.superspec.status</code>	Re-run detection, update the file
Any command that needs a superpowers skill	If the skill was previously detected: false, re-check once (user may have installed it)
User manually edits the file	Respect the manual override — do not overwrite

Why persist this:

1. **Visibility:** Anyone reading `.specify/` can see which superpowers the project uses
2. **Auditability:** The `last_checked` timestamp shows when detection last ran
3. **Speed:** No need to check the filesystem on every command invocation
4. **Override:** Users can manually set `detected: true/false` to force behavior

Reading superpowers status during resume:

When the resume check runs, it reads `superpowers.yml` instead of re-detecting. This means a command like `/speckit.superspec.brainstorm` will use the cached detection result to decide between enhanced mode (superpowers) and fallback mode (built-in). If a skill was detected: false last time, the command does a single re-check before falling back — in case the user installed it since the last session.